TEAMSPARK

Software Requirements Specification (SRS)

Version 1.0

Automated Web-Based Hackathon & Project Team Formation System

Document Item	Details
Project	TeamSpark – Automated Hackathon & Project Team Formation System
Version	1.0
Document Type	Software Requirements Specification
Primary Users	Students and Administrators
Platforms	Web application
Status	Project / Academic Specification
Submitted By	Bhavya Bansal (2427030421) Raj Shukla (2427030683)

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of TeamSpark, a web-based application that helps students find and form teams for hackathons, internships, and academic group projects. It covers listing creation, automated candidate suggestion, join requests, and basic administrative moderation. Integration with official college placement or hackathon-registration systems, and real-time in-app chat, are out of scope for this release.

1.2 Problem Statement

Whenever a professor announces a group project or a hackathon opens registration, students go looking for teammates the same way every time — a message in the class WhatsApp group asking if anyone with a certain skill is free, followed by a scramble of replies, missed messages, and people who have already found a team but never said so. There is no way to see, at a glance, who in the batch is still looking for a team, what skills they bring, or what they are hoping to work on. This works reasonably well in a small class but breaks down quickly for anything larger, such as a department-wide hackathon, where the pool of possible teammates is much bigger than any one WhatsApp group can cover. TeamSpark addresses this by letting students post listings of what they are working on and what skills they still need, automatically suggesting well-matched candidates based on skill, interest, and availability, and letting students send and accept join requests directly within the platform.

1.3 Document Conventions

Functional requirements are tagged REQ-TS-number and prioritized High, Medium, or Low. High-priority requirements are required for V1.0. “Student” denotes any registered user who may create a listing or apply to one; “Admin” denotes a user with moderation privileges. “Listing” refers to a posted team opening, and “Compatibility Score” refers to the numeric match calculated by the system between a student and a listing.

1.4 Intended Audience and Reading Suggestions

This document is intended for developers, UI/UX designers, QA testers, project evaluators, and other stakeholders. Developers should read Sections 2–5 in full. Testers should focus on the functional and non-functional requirements. Readers seeking a high-level understanding should begin with Sections 1 and 2.

1.5 Product Scope

A Student registers, builds a profile listing their skills and availability, and can either create a Listing for a project or hackathon team they are forming, or search and browse existing Listings. The system automatically calculates a Compatibility Score between a Student and an open Listing based on skill match, interest overlap, and availability, and surfaces the top-ranked candidates to the Listing owner. The Listing owner reviews these suggestions and sends or

accepts join requests accordingly. Administrators can moderate listings, manage users, and view basic activity reports.

1.6 References

- [1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.
- [2] K. E. Wiegers, Software Requirements Specification Template, 1999.
- [3] Informal observation of team-formation practices within the department, primarily through class WhatsApp groups and hackathon sign-up sheets.

2. System Scope

2.1 In-Scope Features

- User registration, login, and profile management, including skills, interests, and availability.
- Creation and management of team listings by students, including required skills and team size.
- Search and filtering of listings by skill, interest, and availability.
- Automated calculation of a Compatibility Score between a student and an open listing.
- Display of top-ranked suggested candidates to a listing owner.
- Sending, accepting, and rejecting join requests.
- Basic notifications for new requests, acceptances, and rejections.
- Listing status management (Open, Full, Closed).
- Admin moderation of listings and users, and basic activity reports.

2.2 Out-of-Scope Features for V1.0

- Integration with official college placement or hackathon-registration systems.
- Real-time in-app chat or file sharing between matched teammates.
- Automated project submission, grading, or evaluation of team output.
- Payment or prize-distribution handling for hackathons.
- AI-based resume parsing to auto-fill a student's skill profile.

3. System Environment and Resources

3.1 Software Requirements

Component	Recommended Requirement
Client Platform	Responsive web application, accessible through modern browsers
Frontend	React / HTML-CSS-JS
Backend	REST API using Node.js/Express, Java/Spring Boot, or equivalent
Database	MySQL/PostgreSQL or equivalent relational database
Authentication	Secure session/token-based authentication
Matching Logic	Server-side scoring service computing skill, interest, and availability match
Notifications	Email/SMS service for requests and updates
Development Tools	VS Code / IntelliJ or equivalent
Testing	Unit, integration, UI, API, and performance testing tools

3.2 Hardware Requirements

Hardware	Minimum / Suggested Requirement
Student Device	Computer or laptop with a modern web browser and internet access
Admin Device	Computer with a modern web browser and internet access
Server	Cloud/VPS server capable of running the web application and database services
Network	Wi-Fi/broadband or equivalent reliable internet connection
Development Machine	Computer with at least 8 GB RAM and sufficient storage for IDE/tools

4. UML Diagrams

4.1 Use Case Diagram

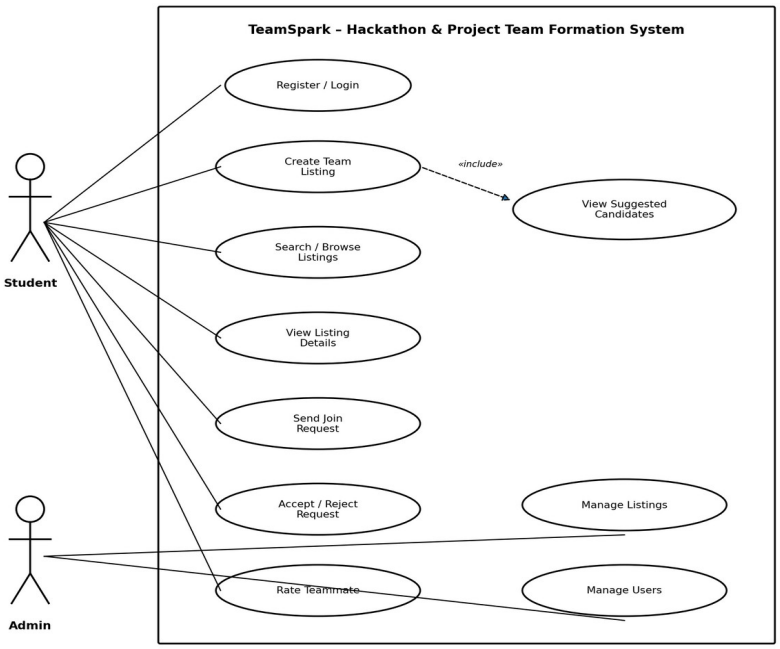


Figure 4.1: Use Case Diagram – TeamSpark

4.2 Class Diagram

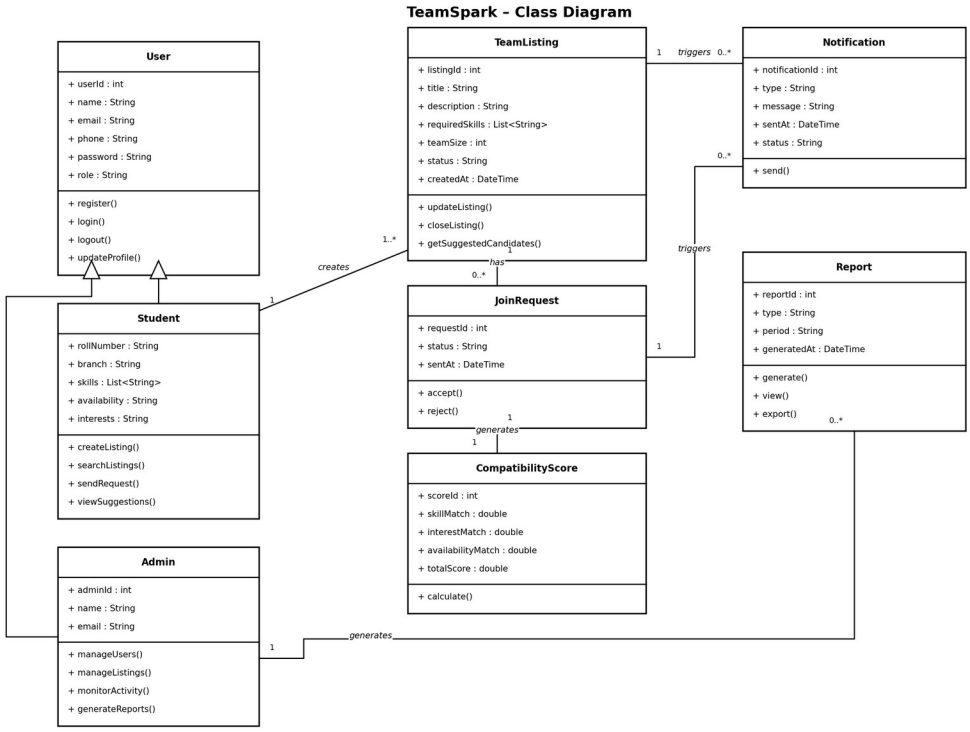


Figure 4.2: Class Diagram – TeamSpark

4.3 Object Diagram

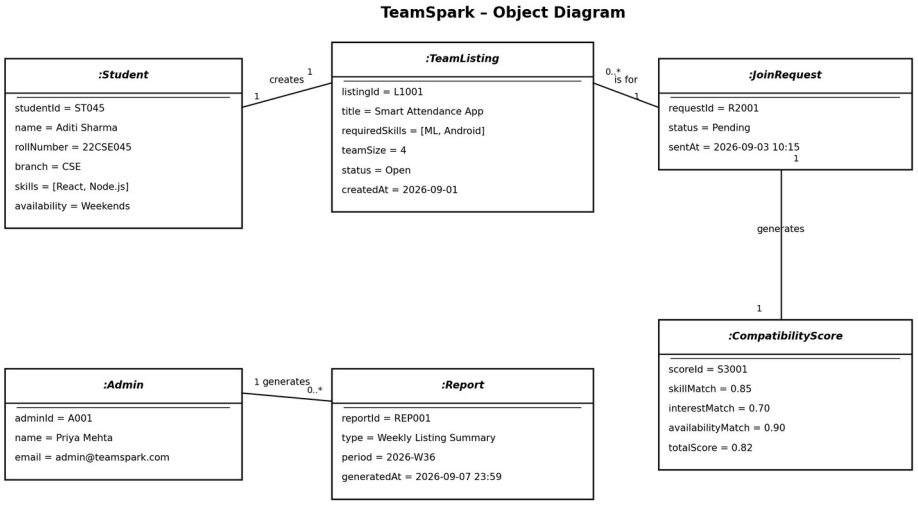


Figure 4.3: Object Diagram – TeamSpark

4.4 Package Diagram

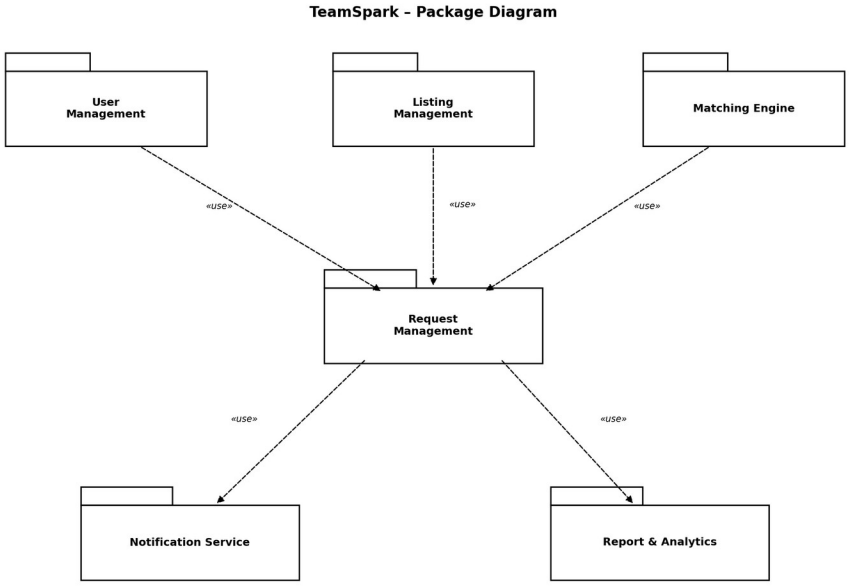


Figure 4.4: Package Diagram – TeamSpark

4.5 Component Diagram

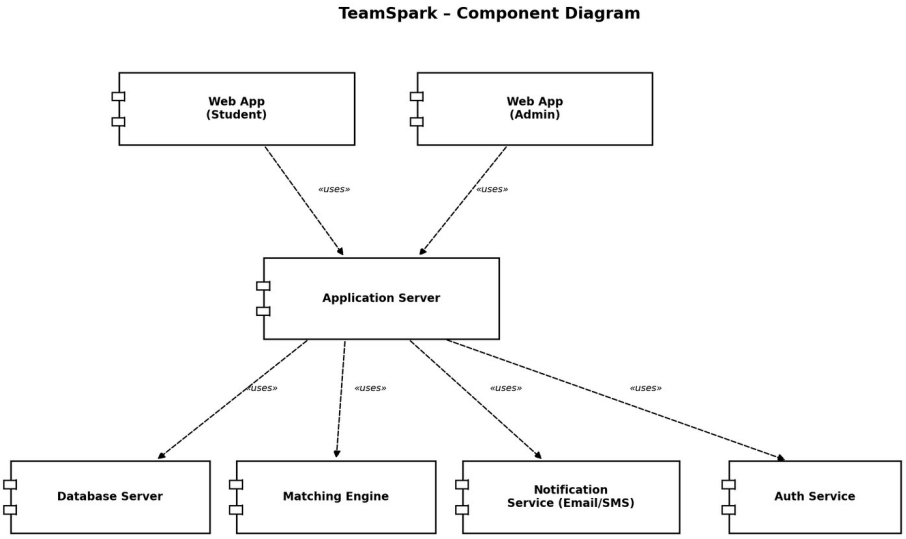


Figure 4.5: Component Diagram – TeamSpark

4.6 Deployment Diagram

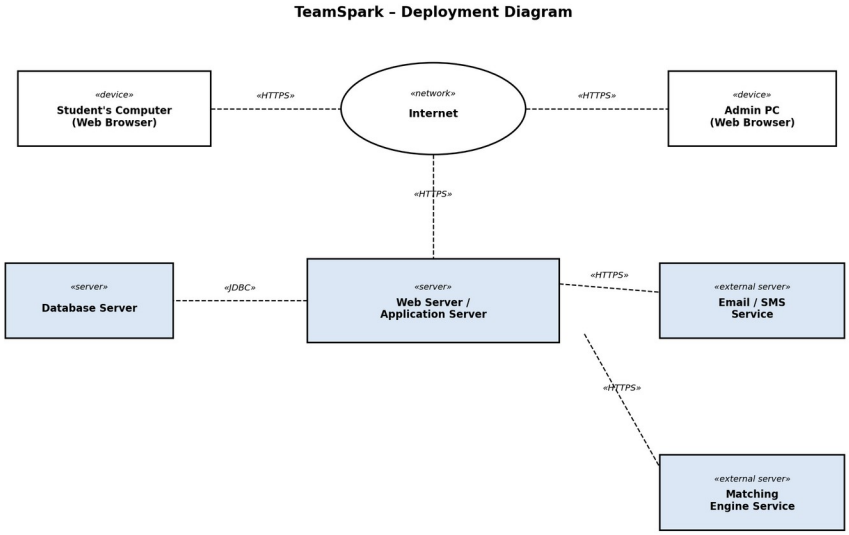


Figure 4.6: Deployment Diagram – TeamSpark

5. Functional Requirements

ID	Function	Requirement	Priority
REQ-TS-01	User Registration	The system shall allow a new Student to create an account using required identity/contact information.	High
REQ-TS-02	User Login	The system shall authenticate registered users and provide role-appropriate access.	High
REQ-TS-03	Profile Management	The system shall allow students to view and update their skills, interests, and availability.	High
REQ-TS-04	Create Listing	The system shall allow a Student to create a team listing with a title, description, required skills, and team size.	High
REQ-TS-05	Listing Management	The system shall allow a Student to edit, close, or reopen their own listings.	High
REQ-TS-06	Search Listings	The system shall allow Students to search and filter listings by skill, interest, and availability.	High
REQ-TS-07	Compatibility Scoring	The system shall automatically calculate a Compatibility Score between a Student and an open Listing based on skill match, interest overlap, and availability.	High
REQ-TS-08	Suggested Candidates	The system shall display the top-ranked candidates for a listing to the listing owner, ordered by Compatibility Score.	High
REQ-TS-09	Send Join Request	The system shall allow a Student to send a join request for an open listing.	High
REQ-TS-10	Accept/Reject Request	The system shall allow a listing owner to accept or reject an incoming join request.	High
REQ-TS-11	Listing Status Update	The system shall update a listing's status automatically once its required team size is reached.	Medium
REQ-TS-12	Notifications	The system shall notify students of new join	Medium

ID	Function	Requirement	Priority
		requests, acceptances, and rejections.	
REQ-TS-13	Booking/Request History	The system shall allow students to view their sent and received requests, past and active.	Medium
REQ-TS-14	Admin – Manage Listings	The system shall allow an Admin to review and remove inappropriate or duplicate listings.	Medium
REQ-TS-15	Admin – Manage Users	The system shall allow an Admin to view and manage registered user accounts.	Medium
REQ-TS-16	Admin Reports	The system shall allow an Admin to generate basic reports on listing and matching activity.	Low

6. Non-Functional Requirements

ID	Category	Requirement
NFR-TS-01	Performance	Common search and matching operations should normally return within 3 seconds under expected load.
NFR-TS-02	Availability	The service should be available throughout normal usage periods with graceful handling of temporary failures.
NFR-TS-03	Security	Passwords, authentication tokens, and personal profile data shall be protected using appropriate security controls.
NFR-TS-04	Privacy	The system shall collect only necessary profile information and restrict visibility according to user roles.
NFR-TS-05	Reliability	Listings, requests, and compatibility scores shall be stored consistently and should not be lost during normal failures.
NFR-TS-06	Usability	A first-time student should be able to create a listing or send a join request through a simple, understandable interface.
NFR-TS-07	Scalability	The backend architecture should support increasing users and listings without major redesign.
NFR-TS-08	Maintainability	The software should use modular components, documented APIs, and a clearly separated matching-logic module.
NFR-TS-09	Compatibility	The system should be accessible through major web browsers on both desktop and mobile devices, without requiring a dedicated app.
NFR-TS-10	Accessibility	Interfaces should use readable text, clear controls, sufficient contrast, and meaningful error messages.
NFR-TS-11	Fairness/Transparency	The system shall show a listing owner the basis of a suggested match (e.g., matched skills) rather than only a raw score.
NFR-TS-12	Auditability	Important listing, request, and administrative actions should be logged for operational troubleshooting.